

Cours 04 — Pull requests et collaboration

Module 4/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - expliquer le rôle d'une **pull request** (PR) : proposer, discuter et valider du code **avant** de le fusionner ; - mener les deux grands workflows — **branche partagée** (en équipe) et **fork** (open source) ; - choisir le bon **mode de fusion** d'une PR (merge, squash, rebase) ; - relier une PR à une **issue**, faire une **revue de code**, et **protéger** la branche main.

Théorie

La pull request : le lieu de la revue

Une pull request est une **demande de fusion** d'une branche vers une autre, enrichie d'un espace de discussion. Ce n'est pas une commande Git : c'est une fonctionnalité de GitHub, construite **par-dessus** git merge. Autour de la fusion, la PR ajoute tout ce qui manque à un merge en ligne de commande :

- une **description** qui explique l'intention du changement ;
- le **diff commenté ligne par ligne** — on peut discuter une ligne précise ;
- les **revues** (approbation, demandes de changement) ;
- les **statuts de CI** (les tests du cours 06 passent-ils ?).

Sur un projet sérieux, on ne fusionne **jamais** directement dans main : tout passe par une PR. C'est là que le code est lu, discuté et validé avant d'entrer dans la version stable.

Deux workflows selon vos droits

Workflow	Contexte	Principe
Branche partagée	Vous avez accès en écriture au dépôt (votre projet, votre équipe)	Vous poussez une branche feature/x sur le dépôt commun, puis ouvrez une PR interne
Fork	Vous n'avez pas l'accès en écriture (contribuer à un projet open source)	Vous forkez (copie personnelle), poussez sur votre fork, puis ouvrez une PR du fork vers l'original

Le fork ajoute une notion : deux distants au lieu d'un.

upstream = le dépôt ORIGINAL (celui auquel vous contribuez, lecture seule pour vous)

origin = VOTRE fork (votre copie, où vous poussez)

upstream —fork→ origin —push→ (votre branche) —PR→ upstream

On synchronise régulièrement son fork avec upstream (git fetch upstream) pour ne pas diverger de l'original pendant qu'on travaille.

Les trois modes de fusion d'une PR

Au moment de fusionner, GitHub propose trois boutons. Ils produisent des historiques différents :

Mode	Ce qu'il fait	Quand le choisir
Merge commit	Garde tous les commits de la branche + un commit de fusion	Conserver le détail du travail
Squash and merge	Écrase toute la PR en un seul commit propre	Le plus courant : main reste net
Rebase and merge	Rejoue les commits un à un, sans commit de fusion	Historique linéaire, sans écraser

En pratique, **squash and merge** est le choix par défaut de beaucoup d'équipes : peu importe si votre branche a dix commits « wip », « fix typo », « oups » — main ne verra qu'un seul commit propre, avec le titre de la PR comme message.

Issues et protection de branche

Les **issues** tracent les bugs, les idées, les tâches. On les relie aux PR avec des mots-clés dans la description : écrire `Fixes #12` dans une PR **ferme automatiquement** l'issue n° 12 dès que la PR est fusionnée. (Autres mots reconnus : `Closes`, `Resolves`.)

Les **règles de protection** de main verrouillent la branche stable. Configuration typique :

- interdiction de pousser directement sur main (PR obligatoire) ;
- au moins **une revue approuvée** avant fusion ;
- **CI verte** exigée (les tests doivent passer).

Résultat : personne, pas même vous, ne peut glisser du code non relu ni non testé dans main.

Commandes commentées

Workflow branche partagée

```
git switch -c feature/export-csv      # créer la branche de travail
# ... des commits ...
git push -u origin feature/export-csv # la pousser sur le dépôt commun
# → GitHub affiche un bandeau « Compare & pull request » : cliquez pour ouvrir la PR
```

Vous poussez votre branche sur origin, puis GitHub détecte la nouvelle branche et propose d'ouvrir la PR d'un clic. Le corps de la PR se rédige dans l'interface web.

Workflow fork (open source)

```
# 1) Bouton « Fork » sur GitHub (crée VOTRE copie), puis on clone SON fork :
git clone git@github.com:VOTRE-PSEUDO/projet.git
cd projet

# 2) Ajouter le dépôt ORIGINAL comme second distant, nommé « upstream »
git remote add upstream https://github.com/AUTEUR/projet.git
git remote -v                # origin = votre fork ; upstream = l'original

# 3) Rester à jour avec l'original pendant qu'on travaille
git fetch upstream
git rebase upstream/main      # rejouer son travail par-dessus la dernière version
```

La distinction clé : `origin` est **votre** fork (vous y poussez), `upstream` est l'**original** (vous en tirez, vous n'y poussez pas). On se rebase régulièrement sur `upstream/main` pour que la PR reste propre et fusionnable.

Mettre à jour une PR déjà ouverte

```
# ... on corrige suite à une revue ...
git add . && git commit -m "fix: prend en compte la revue"
git push                                # pousser sur la MÊME branche met la PR à
jour
```

Pas besoin de rouvrir une PR : tout nouveau commit poussé sur la branche apparaît automatiquement dans la PR existante. La discussion et l'historique de revue sont conservés.

En raccourci, avec GitHub CLI

```
gh pr create --title "Export CSV des ordres" --body "Fixes #12"
gh pr status                                # état de vos PR (revue demandée, CI, etc.)
gh pr checks                                # statut de la CI de la PR courante
```

La CLI `gh` (détaillée au cours 08) ouvre et suit une PR sans quitter le terminal. `Fixes #12` dans le corps reliera la PR à l'issue 12.

Anatomie d'une bonne PR

Ce n'est pas une commande, mais c'est ce qui distingue une PR relue en cinq minutes d'une PR qui traîne une semaine :

1. **Petite** — une seule intention, idéalement moins de ~400 lignes modifiées. Une grosse PR est une PR mal relue.
2. **Titre à l'impératif**, description qui explique le **pourquoi**, pas seulement le quoi.
3. **Auto-revue** — relisez votre propre diff dans l'onglet *Files changed* avant de demander une revue. On y attrape la moitié de ses erreurs.
4. **Liée à une issue** quand elle en résout une (`Fixes #N`).

Exercices pratiques

Réutilisez le dépôt poussé au cours 03.

1. **PR complète**. Créez une branche, poussez-la, ouvrez une PR, **commentez vous-même** une ligne du diff (onglet *Files changed*), puis fusionnez en **squash**. Vérifiez ensuite avec `git log --oneline` sur `main` qu'il n'y a bien qu'un seul commit issu de la PR.
2. **Issue liée**. Créez une issue « Ajouter un README », puis une PR dont la description contient `Fixes #1`. Fusionnez et constatez que l'issue se ferme **toute seule**.
3. **Protection**. Activez une règle de protection sur `main` (PR + 1 revue obligatoires), puis tentez un `git push direct` sur `main` : il doit être refusé par le distant.
4. **Fork**. Forkez un petit dépôt public, clonez **votre** fork, ajoutez `upstream`, et synchronisez-vous avec `git fetch upstream`.

Solutions des exercices

Solution 1 — PR complète en squash

```
git switch -c feature/readme
echo "# Gestionnaire d'ordres" > README.md
git add . && git commit -m "docs: ébauche de README"
echo "Petit dépôt de scripts de trading." >> README.md
```

```
git add . && git commit -m "docs: ajoute une description" # 2 commits sur la
branche
git push -u origin feature/readme
```

Sur GitHub : bandeau **Compare & pull request** → rédigez titre + description → **Create pull request**.

Onglet **Files changed** : survolez une ligne, cliquez le + bleu, laissez un commentaire (« je clarifie ici »).

Puis **Merge pull request** → **Squash and merge**. En local :

```
git switch main
git pull
git log --oneline
# * a1b2c3d docs: ajoute un README (#1) ← UN SEUL commit, malgré les deux de la
branche
```

Le squash a fondu les deux commits de la branche en un seul, avec le titre de la PR.

Solution 2 — Issue liée

Sur GitHub : onglet **Issues** → **New issue**, titre « Ajouter un README », **Submit**. Notez son numéro (par ex. #1). Puis :

```
git switch -c feature/readme-lie
echo "# Projet" > README.md
git add . && git commit -m "docs: README"
git push -u origin feature/readme-lie
```

Ouvrez la PR et écrivez dans sa **description** : Fixes #1. Fusionnez la PR. Retournez dans l'onglet **Issues** : l'issue #1 est passée en **Closed** automatiquement. Le mot-clé Fixes (ou Closes, Resolves) suivi du numéro fait le lien.

Solution 3 — Protection de main

Sur GitHub : *Settings* → *Branches* → *Add branch protection rule*. Motif de branche : main. Cochez **Require a pull request before merging** et **Require approvals** (1). Enregistrez. Puis, en local, tentez le push interdit :

```
git switch main
echo "modif directe" >> notes.md
git add . && git commit -m "test: push direct interdit"
git push
# ! [remote rejected] main -> main (protected branch hook declined)
```

Le distant refuse : main n'accepte plus que des fusions de PR relues. Annulez votre commit local de test avec `git reset --hard origin/main` (voir cours 05).

Note : sur un dépôt personnel gratuit, s'auto-approuver peut être impossible ; l'exercice montre surtout le refus du push direct, qui est l'essentiel.

Solution 4 — Fork

Choisissez un petit dépôt public, cliquez **Fork** (en haut à droite). GitHub crée `github.com/VOTRE-PSEUDO/le-projet`. Puis :

```
git clone git@github.com:VOTRE-PSEUDO/le-projet.git # on clone SON fork
cd le-projet

git remote add upstream https://github.com/AUTEUR-ORIGINAL/le-projet.git
git remote -v
# origin      git@github.com:VOTRE-PSEUDO/le-projet.git  (votre fork : push + fetch)
# upstream    https://github.com/AUTEUR-ORIGINAL/...      (l'original : fetch
seulement)
```

```
git fetch upstream          # récupère les nouveautés de l'original
git log upstream/main --oneline  # voir l'historique de l'original
```

Vous êtes prêt à contribuer : brancher depuis `upstream/main`, committer, pousser sur `origin`, ouvrir une PR vers l'original.

Points clés à retenir

- La PR est le **lieu de la revue** : on discute et on valide le code **avant** qu'il n'entre dans `main`.
- **Petites PR = revues rapides = moins de bugs**. Une seule intention par PR.
- **Squash and merge** est le mode par défaut le plus sûr : `main` reste propre, un commit par PR.
- Fixes `#N` dans la description relie et **ferme** l'issue à la fusion.
- Deux distants dans le workflow fork : `origin` (votre fork, on pousse) et `upstream` (l'original, on tire).
- Protégez `main` dès qu'un projet compte plus d'une personne — ou même juste pour vous discipliner.